



آزمایشگاه طراحی سیستم‌های دیجیتال

ترم تابستان ۱۴۰۵

استاد: دکتر انصاری

طراحی UART

آزمایش هفتم

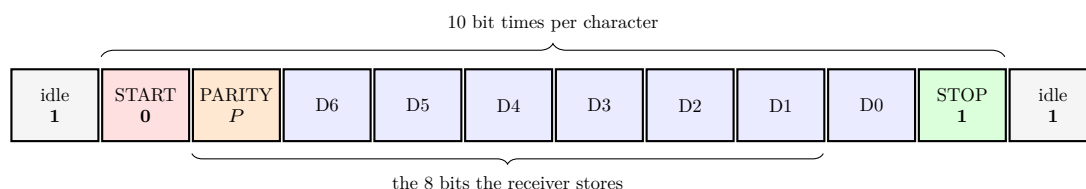
خلاصه

در این آزمایش یک Universal Asynchronous Receiver Transmitter طراحی و پیاده‌سازی شده است. طبق صورت آزمایش، فرستنده هر بار یک کد ASCII هفت‌بیتی را به‌صورت سریال بیرون می‌فرستد و گیرنده پس از دریافت بیت شروع، هشت بیت بعدی — یعنی توازن و داده — را در یک ثبات ۸ بیتی ذخیره می‌کند. در پایان دو واحد کامل به یکدیگر وصل شده‌اند تا نشان داده شود مبادله در هر دو جهت درست انجام می‌شود.

نکته‌ای که تمام طراحی حول آن می‌چرخد این است که UART یک پروتکل ناهم‌گام است: هیچ سیم کلاکی بین دو طرف کشیده نمی‌شود. تنها چیزی که فرستنده و گیرنده در آن توافق دارند نرخ بیت و قالب فریم است. بنابراین گیرنده باید خودش لحظه‌ی شروع فریم را از روی خط تشخیص دهد و سپس بدون کمک گرفتن از کلاک فرستنده، محل درست نمونه‌برداری هر بیت را حدس بزند.

قالب فریم

صورت آزمایش قالب را دقیقاً مشخص کرده است: یک بیت شروع، سپس یک بیت توازن، سپس ۷ بیت داده و در انتها حداقل یک بیت خاتمه — در مجموع ۱۰ بیت.



دو تفاوت با UART استاندارد وجود دارد و هر دو مستقیماً از صورت آزمایش می‌آید:

۱. بیت توازن قبل از داده فرستاده می‌شود، نه بعد از آن. پیامد این انتخاب برای سخت‌افزار مهم است: فرستنده باید تمام کاراکتر را پیش از شروع ارسال در اختیار داشته باشد، چون توازن تابعی از همه‌ی هفت بیت است. به همین دلیل داده به‌صورت موازی بارگذاری می‌شود و توازن در همان لحظه‌ی بارگذاری به‌صورت ترکیبی محاسبه می‌شود.

۲. بار مفید ۷ بیت است و گیرنده ۸ بیت پس از بیت شروع را در یک ثبات ذخیره می‌کند.

چرا داده MSB اول فرستاده می‌شود

در UART استاندارد داده از کم‌ارزش‌ترین بیت شروع می‌شود. اینجا عمداً برعکس عمل شده است و دلیلش خواسته‌ی خود صورت آزمایش است: گیرنده باید ۸ بیت را در یک ثبات ۸ بیتی بریزد.

گیرنده یک ثبات لغزشی است که هر بیت جدید را از سمت کم‌ارزش وارد می‌کند. اگر ترتیب ارسال $P, D_6, D_5, \dots$ باشد، محتوای ثبات پس از هشت شیفต์ دقیقاً این است:

$$rx_reg = \{P, D6, D5, D4, D3, D2, D1, D0\}$$

یعنی بیت توازن در پرارزش‌ترین جایگاه و کد ASCII بدون هیچ جابه‌جایی در هفت بیت پایین. اگر داده LSB اول فرستاده می‌شد، همان ثبات کد ASCII را برعکس نگه می‌داشت و برای استفاده از آن باید یک معکوس‌کننده‌ی بیتی اضافه می‌شد — منطقی که هیچ کاری جز جبران یک انتخاب بد نمی‌کند.

تولید نرخ بیت و نمونه‌برداری ۱۶ برابر

هر واحد یک شمارنده‌ی تقسیم دارد که پالسی به نام `os_tick` با نرخ ۱۶ برابر نرخ بیت تولید می‌کند. فرستنده و گیرنده‌ی یک واحد هر دو از همین یک پالس استفاده می‌کنند، پس هزینه‌ی هر UART یک شمارنده است نه دو تا.

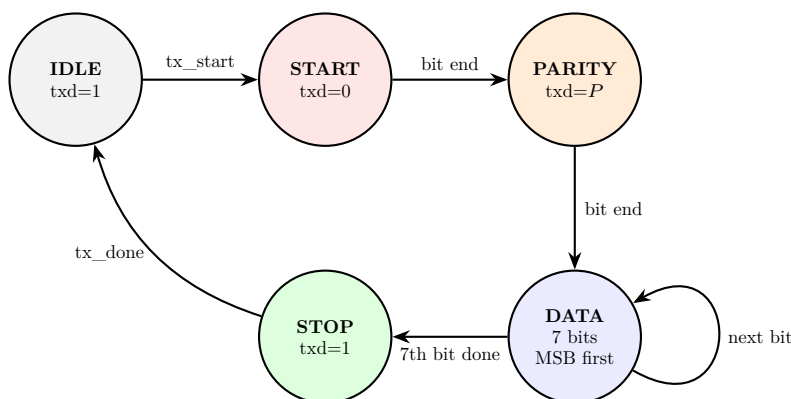
```
DIVISOR = f_clk / (16 * baud) // synthesis: e.g. 50 MHz / (16 * 115200) = 27
```

نسبت تقسیم پارامتر است تا در سنتز مقدار واقعی و در شبیه‌سازی مقداری کوچک استفاده شود؛ در غیر این صورت هر تست‌بنچ باید میلیون‌ها کلاک بی‌کار را شبیه‌سازی می‌کرد.

اما چرا اصلاً ۱۶ برابر؟ چون گیرنده کلاک فرستنده را ندارد. تنها لنگرگاه زمانی‌اش لبه‌ی پایین‌رونده‌ی بیت شروع است و این لبه با دقت یک پالس نمونه‌برداری تشخیص داده می‌شود. با ۱۶ برابر، این خطای اولیه $\frac{1}{16}$ یک بیت است و نقطه‌ی نمونه‌برداری آن قدر به وسط بیت نزدیک می‌ماند که تا انتهای فریم هم داخل بیت درست باقی بماند. جزئیات این حاشیه‌ی خطا در بخش تست‌ها می‌آید.

فرستنده

فرستنده یک ماشین حالت ساده است که هر حالت آن دقیقاً ۱۶ پالس `os_tick` طول می‌کشد:



دو نکته‌ی پیاده‌سازی:

بیت شروع در همان لبه‌ی بارگذاری روی خط می‌رود. اگر منتظر اولین `os_tick` می‌ماندیم، اولین بیت به اندازه‌ی فاز باقی‌مانده‌ی شمارنده کش می‌آمد. با پایین بردن فوری خط و صفر کردن هم‌زمان شمارنده‌ی داخل بیت، طول بیت شروع دقیقاً ۱۶ پالس می‌شود و همه‌ی بیت‌های بعدی هم به همان اندازه.

درخواست جدید در میانه‌ی فریم نادیده گرفته می‌شود، نه صف. سیگنال `tx_busy` برای همین بیرون داده شده است. اگر درخواست‌ها صف می‌شدند، به یک بافر نیاز بود؛ و اگر فریم جاری را قطع می‌کردند، گیرنده‌ی طرف مقابل یک فریم ناقص می‌دید. این رفتار در تست‌بنچ صریحاً بررسی شده است.

```
1 `include "uart_pkg.vh"
2
3 module uart_tx #(
```

```

4  parameter ODD_PARITY = 1'b0
5  ) (
6  input wire          Clk,
7  input wire          RstN,
8  input wire          os_tick,
9  input wire          tx_start,
10 input wire [UART_DATA_BITS-1:0] tx_data,
11 output reg          txd,
12 output wire         tx_busy,
13 output reg          tx_done
14 );
15 localparam [2:0] IDLE   = 3'd0,
16                  START  = 3'd1,
17                  PARITY  = 3'd2,
18                  DATA   = 3'd3,
19                  STOP    = 3'd4;
20
21 reg [2:0]          st;
22 reg [3:0]          os;
23 reg [2:0]          idx;
24 reg [UART_DATA_BITS-1:0] sh;
25 reg               par;
26
27 wire par_of_data = ODD_PARITY ? ~(tx_data) : (tx_data);
28
29 wire bit_end = os_tick && (os == UART_OVERSAMPLE - 1);
30
31 assign tx_busy = (st != IDLE);
32
33 always @(posedge Clk or negedge RstN) begin
34     if (!RstN) begin
35         st      <= IDLE;
36         os      <= 4'd0;
37         idx     <= 3'd0;
38         sh      <= {UART_DATA_BITS{1'b0}};
39         par     <= 1'b0;
40         txd     <= 1'b1;
41         tx_done <= 1'b0;
42     end else begin
43         tx_done <= 1'b0;
44
45         if (st != IDLE && os_tick)
46             os <= bit_end ? 4'd0 : os + 4'd1;
47
48         case (st)
49             IDLE: begin
50                 txd <= 1'b1;
51                 if (tx_start) begin
52                     sh <= tx_data;
53                     par <= par_of_data;
54                     txd <= 1'b0;
55                     os <= 4'd0;
56                     idx <= UART_DATA_BITS - 1;
57                     st <= START;
58                 end
59             end
60
61             START: if (bit_end) begin
62                 txd <= par;
63                 st <= PARITY;
64             end
65
66             PARITY: if (bit_end) begin
67                 txd <= sh[UART_DATA_BITS-1];
68                 sh <= {sh[UART_DATA_BITS-2:0], 1'b0};
69                 st <= DATA;
70             end
71
72             DATA: if (bit_end) begin
73                 if (idx == 3'd0) begin
74                     txd <= 1'b1;
75                     st <= STOP;
76                 end else begin
77                     txd <= sh[UART_DATA_BITS-1];
78                     sh <= {sh[UART_DATA_BITS-2:0], 1'b0};
79                     idx <= idx - 3'd1;
80                 end
81             end
82
83             STOP: if (bit_end) begin
84                 txd <= 1'b1;
85                 tx_done <= 1'b1;
86                 st <= IDLE;
87             end
88
89             default: st <= IDLE;
90         endcase

```

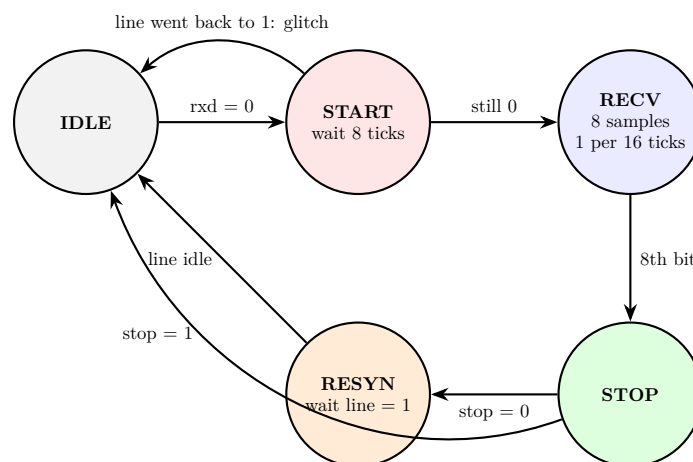
```

91     end
92   end
93 endmodule

```

گیرنده

گیرنده کار سختی دارد: باید بدون کلاک مشترک بفهمد هر بیت کجا شروع و کجا تمام می‌شود.



(الف) هم‌گام‌سازی ورودی. خط سریال از دامنه‌ی زمانی دیگری (یا روی برد، از یک پایه) می‌آید و نباید مستقیم به ماشین حالت وصل شود. دو فلیپ‌فلاپ پشت سر هم احتمال metastability را عملاً از بین می‌برند.

(ب) رد کردن نویز به جای شروع فریم. با پایین آمدن خط، گیرنده نیم بیت صبر می‌کند و دوباره خط را می‌بیند. اگر خط بالا رفته باشد، آنچه دیده شده یک سوزن نویزی بوده و گیرنده به IDLE برمی‌گردد. اگر هنوز پایین باشد، این واقعا بیت شروع است — و از قضا این لحظه دقیقاً وسط بیت شروع است.

(ج) نمونه‌برداری از وسط هر بیت. از همان نقطه به بعد، هر ۱۶ پالس یک بار نمونه گرفته می‌شود. چون نقطه‌ی شروع وسط بیت شروع بود، همه‌ی نمونه‌های بعدی هم دقیقاً وسط بیت خودشان می‌افتند — بیشترین فاصله‌ی ممکن از هر دو لبه، یعنی بیشترین تحمل نسبت به اختلاف نرخ بیت دو طرف.

(د) بازگشت به هم‌گامی پس از خطای فریم. اگر بیت خاتمه صفر خوانده شود، فریم خراب بوده و خط هنوز پایین است. برگشتن مستقیم به IDLE یعنی همان صفر بلافاصله به عنوان بیت شروع بعدی تفسیر شود و یک فریم زباله‌ی دیگر تولید کند. به همین دلیل حالت RESYN اضافه شده که تا بی‌کار شدن خط منتظر می‌ماند. این رفتار هم در تست‌بنچ بررسی شده است.

```

1  `include "uart_pkg.vh"
2
3  module uart_rx #(
4      parameter ODD_PARITY = 1'b0
5  ) (
6      input  wire          Clk,
7      input  wire          RstN,
8      input  wire          os_tick,
9      input  wire          rx_d,
10
11     output reg  [`UART_RX_REG_BITS-1:0] rx_reg,
12     output wire  [`UART_DATA_BITS-1:0]  rx_data,
13     output reg  rx_valid,
14     output reg  parity_err,
15     output reg  frame_err,
16     output wire  rx_busy
17 );
18     localparam integer OS = `UART_OVERSAMPLE;
19
20     localparam [2:0] IDLE = 3'd0,

```

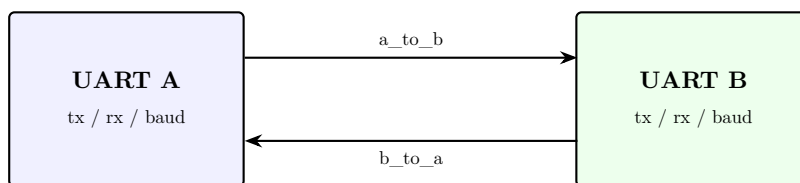
```

21         START = 3'd1,
22         RECV  = 3'd2,
23         STOP  = 3'd3,
24         RESYN = 3'd4;
25
26     reg rx_s0, rx_s1;
27     always @(posedge Clk or negedge RstN) begin
28         if (!RstN) {rx_s0, rx_s1} <= 2'b11;
29         else       {rx_s0, rx_s1} <= {rx_d, rx_s0};
30     end
31
32     reg [2:0] st;
33     reg [3:0] os;
34     reg [2:0] nbit;
35     reg [`UART_RX_REG_BITS-1:0] sh;
36
37     assign rx_data = rx_reg[`UART_DATA_BITS-1:0];
38     assign rx_busy = (st != IDLE);
39
40     wire exp_par = ODD_PARITY ? ~(^sh[`UART_DATA_BITS-1:0]) : (^sh[`UART_DATA_BITS-1:0]);
41
42     always @(posedge Clk or negedge RstN) begin
43         if (!RstN) begin
44             st <= IDLE;
45             os <= 4'd0;
46             nbit <= 3'd0;
47             sh <= {`UART_RX_REG_BITS{1'b0}};
48             rx_reg <= {`UART_RX_REG_BITS{1'b0}};
49             rx_valid <= 1'b0;
50             parity_err <= 1'b0;
51             frame_err <= 1'b0;
52         end else begin
53             rx_valid <= 1'b0;
54
55             case (st)
56                 IDLE: if (!rx_s1) begin
57                     os <= 4'd0;
58                     st <= START;
59                 end
60
61                 START: if (os_tick) begin
62                     if (os == OS/2 - 1) begin
63                         if (rx_s1) begin
64                             st <= IDLE;
65                         end else begin
66                             os <= 4'd0;
67                             nbit <= 3'd0;
68                             st <= RECV;
69                         end
69                     end else begin
70                         os <= os + 4'd1;
71                     end
72                 end
73             end
74
75             RECV: if (os_tick) begin
76                 if (os == OS-1) begin
77                     os <= 4'd0;
78                     sh <= {sh[`UART_RX_REG_BITS-2:0], rx_s1};
79                     nbit <= nbit + 3'd1;
80                     if (nbit == `UART_RX_REG_BITS - 1) st <= STOP;
81                 end else begin
82                     os <= os + 4'd1;
83                 end
84             end
85
86             STOP: if (os_tick) begin
87                 if (os == OS-1) begin
88                     os <= 4'd0;
89                     rx_reg <= sh;
90                     parity_err <= (sh[`UART_RX_REG_BITS-1] != exp_par);
91                     frame_err <= ~rx_s1;
92                     rx_valid <= 1'b1;
93                     st <= rx_s1 ? IDLE : RESYN;
94                 end else begin
95                     os <= os + 4'd1;
96                 end
97             end
98
99             RESYN: if (rx_s1) st <= IDLE;
100
101             default: st <= IDLE;
102         endcase
103     end
104 end
105 endmodule

```

اتصال دو واحد

خواسته‌ی پایانی صورت آزمایش این است که دو واحد به یکدیگر وصل شوند. ماژول uart_link دقیقاً همین است: خروجی فرستنده‌ی هر واحد به ورودی گیرنده‌ی واحد دیگر می‌رود و این دو سیم تمام «کابل» بین دو دستگاه‌اند.



no shared clock – only the agreed baud rate and frame format

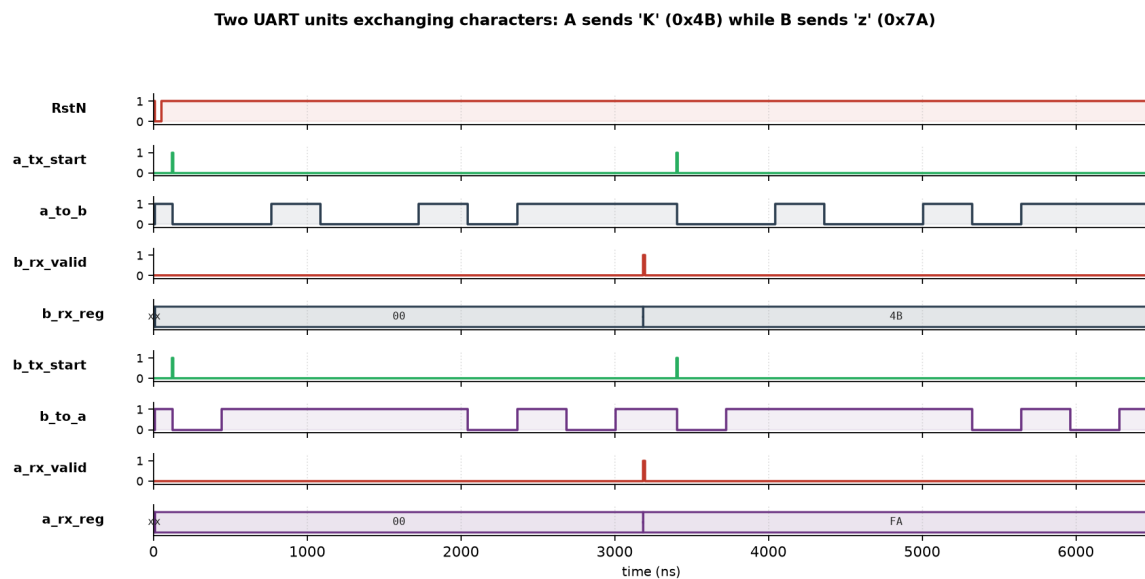
دو واحد پارامتر تقسیم جداگانه دارند. با مقادیر برابر، لینک کاملاً هم‌نرخ است؛ با مقادیر کمی متفاوت، دو بردی مدل می‌شوند که کریستال‌هایشان با هم اختلاف دارند — و این دقیقاً همان چیزی است که نمونه‌برداری ۱۶ برابر باید از پس آن بربایید.

```

1  `include "uart_pkg.vh"
2
3  module uart_link #(
4      parameter integer A_OS_DIVISOR = 16,
5      parameter integer B_OS_DIVISOR = 16,
6      parameter        ODD_PARITY    = 1'b0
7  ) (
8      input  wire          Clk,
9      input  wire          RstN,
10
11     // unit A
12     input  wire          a_tx_start,
13     input  wire [ `UART_DATA_BITS-1:0] a_tx_data,
14     output wire          a_tx_busy,
15     output wire          a_tx_done,
16     output wire [ `UART_RX_REG_BITS-1:0] a_rx_reg,
17     output wire [ `UART_DATA_BITS-1:0] a_rx_data,
18     output wire          a_rx_valid,
19     output wire          a_parity_err,
20     output wire          a_frame_err,
21
22     // unit B
23     input  wire          b_tx_start,
24     input  wire [ `UART_DATA_BITS-1:0] b_tx_data,
25     output wire          b_tx_busy,
26     output wire          b_tx_done,
27     output wire [ `UART_RX_REG_BITS-1:0] b_rx_reg,
28     output wire [ `UART_DATA_BITS-1:0] b_rx_data,
29     output wire          b_rx_valid,
30     output wire          b_parity_err,
31     output wire          b_frame_err,
32
33     output wire          a_to_b,
34     output wire          b_to_a
35 );
36     uart #(.OS_DIVISOR(A_OS_DIVISOR), .ODD_PARITY(ODD_PARITY)) u_a (
37         .Clk(Clk), .RstN(RstN),
38         .tx_start(a_tx_start), .tx_data(a_tx_data),
39         .txd(a_to_b), .tx_busy(a_tx_busy), .tx_done(a_tx_done),
40         .rxd(b_to_a),
41         .rx_reg(a_rx_reg), .rx_data(a_rx_data), .rx_valid(a_rx_valid),
42         .parity_err(a_parity_err), .frame_err(a_frame_err),
43         .rx_busy(), .os_tick()
44     );
45
46     uart #(.OS_DIVISOR(B_OS_DIVISOR), .ODD_PARITY(ODD_PARITY)) u_b (
47         .Clk(Clk), .RstN(RstN),
48         .tx_start(b_tx_start), .tx_data(b_tx_data),
49         .txd(b_to_a), .tx_busy(b_tx_busy), .tx_done(b_tx_done),
50         .rxd(a_to_b),
51         .rx_reg(b_rx_reg), .rx_data(b_rx_data), .rx_valid(b_rx_valid),
52         .parity_err(b_parity_err), .frame_err(b_frame_err),
53         .rx_busy(), .os_tick()
54     );
55 endmodule

```

شکل موج



در این شکل موج، واحد A کاراکتر 'K' (کد 0x4B) و واحد B در همان لحظه کاراکتر 'z' (کد 0x7A) را می‌فرستد. دو سیم `a_to_b` و `b_to_a` هم‌زمان مشغول‌اند، یعنی لینک واقعا دوطرفه‌ی هم‌زمان است. مقادیر ذخیره‌شده در ثبات‌ها نشان می‌دهد ترتیب بیت‌ها درست است:

کاراکتر	کد ASCII	توازن زوج	محتوای ثبات ۸ بیتی
'K'	1001011 = 0x4B	•	0x4B = 0_1001011
'z'	1111010 = 0x7A	\	0xFA = 1_1111010

توجه کنید که برای 'K' تعداد یک‌ها زوج است پس بیت توازن صفر می‌شود و محتوای ثبات با خود کد ASCII یکی درمی‌آید؛ برای 'z' تعداد یک‌ها فرد است، بیت توازن یک می‌شود و در پرارزش‌ترین جایگاه ثبات می‌نشیند. در هر دو حالت هفت بیت پایین ثبات بدون هیچ پردازش اضافه همان کد ASCII است — همان چیزی که در بخش ترتیب بیت‌ها درباره‌اش صحبت شد.

تست‌بنچ‌ها

تست‌بنچ	روش
tb_uart_tx	الگوی ۱۰ بیتی مورد انتظار را از روی صورت آزمایش می‌سازد و خط را وسط هر بیت نمونه‌برداری می‌کند
tb_uart_rx	فریم‌ها را بیت‌به‌بیت از داخل خود تست‌بنچ روی خط می‌راند
tb_uart_link	تنها جایی که دو ماژول با هم کار می‌کنند؛ اینجا هدف بررسی مبادله است نه قالب فریم

تست فرستنده هر ۱۲۸ کد ASCII را با هر دو قرارداد توازن زوج و فرد می‌فرستد و هر ۱۰ بیت هر فریم را جداگانه بررسی می‌کند. علاوه بر آن: بی‌کار بودن خط روی مقدار یک، درست بودن `tx_busy` در طول فریم، پالس `tx_done` در انتها، و نادیده گرفته شدن درخواست ارسال در میانه‌ی فریم.

```

1 `timescale 1ns/1ps
2 `include "uart_pkg.vh"
3
4 module tb_uart_tx;
5     localparam integer OSD = 3;
6     localparam integer OS = `UART_OVERSAMPLE;
7
8     reg Clk = 1'b0;
9     reg RstN = 1'b1;
10    reg tx_start = 1'b0;
11    reg [`UART_DATA_BITS-1:0] tx_data = 7'd0;
12
13    wire os_tick;
14    wire txd_e, busy_e, done_e;
15    wire txd_o, busy_o, done_o;
16
17    baud_gen #(.DIVISOR(OSD)) u_baud (.Clk(Clk), .RstN(RstN), .tick(os_tick));
18
19    uart_tx #(.ODD_PARITY(1'b0)) dut_even (
20        .Clk(Clk), .RstN(RstN), .os_tick(os_tick),
21        .tx_start(tx_start), .tx_data(tx_data),
22        .txd(txd_e), .tx_busy(busy_e), .tx_done(done_e)
23    );
24
25    uart_tx #(.ODD_PARITY(1'b1)) dut_odd (
26        .Clk(Clk), .RstN(RstN), .os_tick(os_tick),
27        .tx_start(tx_start), .tx_data(tx_data),
28        .txd(txd_o), .tx_busy(busy_o), .tx_done(done_o)
29    );
30
31    always #5 Clk = ~Clk;
32
33    integer ntick = 0;
34    always @(posedge Clk) if (os_tick) ntick = ntick + 1;
35
36    integer pass = 0, fail = 0;
37    integer c, i, t0;
38    reg [9:0] exp;
39    reg got_done_e, got_done_o;
40
41    always @(posedge Clk) begin
42        if (done_e) got_done_e <= 1'b1;
43        if (done_o) got_done_o <= 1'b1;
44    end
45
46    task automatic chk;
47        input [255:0] name;
48        input ok;
49        begin
50            if (ok) pass = pass + 1;
51            else begin
52                fail = fail + 1;
53                $display(" FAIL @%0t: %0s", $time, name);
54            end
55        end
56    endtask
57
58    task automatic wait_tick;
59        input integer target;
60        begin
61            while (ntick < target) @(posedge Clk);
62            @(negedge Clk);
63        end
64    endtask
65
66    function [9:0] frame;
67        input [`UART_DATA_BITS-1:0] d;
68        input odd;
69        begin
70            frame = {1'b0, odd ? ~(^d) : (^d), d[6], d[5], d[4], d[3], d[2], d[1], d[0], 1'b1};
71        end
72    endfunction
73
74    initial begin
75        got_done_e = 1'b0;
76        got_done_o = 1'b0;
77
78        @(negedge Clk); RstN = 1'b0;
79        repeat (4) @(negedge Clk); RstN = 1'b1;
80        repeat (2) @(negedge Clk);
81
82        chk("line idles high (even)", txd_e === 1'b1);
83        chk("line idles high (odd)", txd_o === 1'b1);
84        chk("not busy after reset", busy_e === 1'b0 && busy_o === 1'b0);
85
86        for (c = 0; c < 128; c = c + 1) begin

```



```

87      @(negedge Clk); tx_data = c[6:0]; tx_start = 1'b1;
88      @(negedge Clk); tx_start = 1'b0;
89      t0 = ntick;
90
91      got_done_e = 1'b0;
92      got_done_o = 1'b0;
93
94      for (i = 0; i < `UART_FRAME_BITS; i = i + 1) begin
95          wait_tick(t0 + OS*i + OS/2);
96          exp = frame(c[6:0], 1'b0);
97          chk("even-parity frame bit", txd_e === exp[9-i]);
98          exp = frame(c[6:0], 1'b1);
99          chk("odd-parity frame bit", txd_o === exp[9-i]);
100         if (i < `UART_FRAME_BITS - 1)
101             chk("busy during frame", busy_e === 1'b1 && busy_o === 1'b1);
102     end
103
104     wait_tick(t0 + OS*`UART_FRAME_BITS + 1);
105     chk("tx_done pulsed", got_done_e && got_done_o);
106     chk("idle again", busy_e === 1'b0 && busy_o === 1'b0);
107     chk("line back to idle 1", txd_e === 1'b1 && txd_o === 1'b1);
108 end
109
110 @(negedge Clk); tx_data = 7'h55; tx_start = 1'b1;
111 @(negedge Clk); tx_start = 1'b0;
112 t0 = ntick;
113 wait_tick(t0 + OS + OS/2);
114 @(negedge Clk); tx_data = 7'h2A; tx_start = 1'b1;
115 @(negedge Clk); tx_start = 1'b0;
116 for (i = 1; i < `UART_FRAME_BITS; i = i + 1) begin
117     wait_tick(t0 + OS*i + OS/2);
118     exp = frame(7'h55, 1'b0);
119     chk("frame unaffected by mid-frame request", txd_e === exp[9-i]);
120 end
121
122 $display("tb_uart_tx: Passed: %0d, Failed: %0d", pass, fail);
123 if (fail != 0) $fatal(1);
124 $finish;
125 end
126 endmodule

```

تست گیرنده همان ۱۲۸ کد را با هر دو قرارداد توازن روی خط می‌راند و علاوه بر آن چهار وضعیت غیرعادی را می‌سازد: بیت توازن خراب، بیت خاتمه‌ی گم‌شده، یک سوزن کوتاه روی خط، و فریم‌های پشت سر هم بدون هیچ فاصله‌ی بی‌کاری.

نکته‌ی ظریفی که این تست روشن می‌کند: پس از خطای توازن، داده همچنان در ثبات می‌نشیند و فقط پرچم خطا بالا می‌رود. تصمیم درباره‌ی دور انداختن یا نینداختن کاراکتر خراب کار لایه‌ی بالاتر است، نه کار UART.

```

1  `timescale ins/1ps
2  `include "uart_pkg.vh"
3
4  module tb_uart_rx;
5      localparam integer OSD = 3;
6      localparam integer OS = `UART_OVERSAMPLE;
7
8      reg Clk = 1'b0, RstN = 1'b1, line = 1'b1;
9      wire os_tick;
10
11     wire [`UART_RX_REG_BITS-1:0] reg_e, reg_o;
12     wire [`UART_DATA_BITS-1:0] dat_e, dat_o;
13     wire vld_e, per_e, fer_e, bsy_e;
14     wire vld_o, per_o, fer_o, bsy_o;
15
16     baud_gen #(.DIVISOR(OSD)) u_baud (.Clk(Clk), .RstN(RstN), .tick(os_tick));
17
18     uart_rx #(.ODD_PARITY(1'b0)) dut_even (
19         .Clk(Clk), .RstN(RstN), .os_tick(os_tick), .rxd(line),
20         .rx_reg(reg_e), .rx_data(dat_e), .rx_valid(vld_e),
21         .parity_err(per_e), .frame_err(fer_e), .rx_busy(bsy_e)
22     );
23
24     uart_rx #(.ODD_PARITY(1'b1)) dut_odd (
25         .Clk(Clk), .RstN(RstN), .os_tick(os_tick), .rxd(line),
26         .rx_reg(reg_o), .rx_data(dat_o), .rx_valid(vld_o),
27         .parity_err(per_o), .frame_err(fer_o), .rx_busy(bsy_o)
28     );
29
30     always #5 Clk = ~Clk;
31
32     integer ntick = 0;
33     always @(posedge Clk) if (os_tick) ntick = ntick + 1;
34

```

```

35 integer pass = 0, fail = 0;
36 integer c, i;
37
38 reg                                seen_e, seen_o;
39 reg [`UART_RX_REG_BITS-1:0] cap_e, cap_o;
40 reg                                cpe_e, cfe_e, cpe_o, cfe_o;
41
42 always @(posedge Clk) begin
43     if (vld_e) begin seen_e <= 1'b1; cap_e <= reg_e; cpe_e <= per_e; cfe_e <= fer_e; end
44     if (vld_o) begin seen_o <= 1'b1; cap_o <= reg_o; cpe_o <= per_o; cfe_o <= fer_o; end
45 end
46
47 task automatic chk;
48     input [255:0] name;
49     input ok;
50     begin
51         if (ok) pass = pass + 1;
52         else begin
53             fail = fail + 1;
54             $display(" FAIL %0t: %0s", $time, name);
55         end
56     end
57 endtask
58
59 task automatic hold_ticks;
60     input integer n;
61     integer target;
62     begin
63         target = ntick + n;
64         while (ntick < target) @(posedge Clk);
65         @(negedge Clk);
66     end
67 endtask
68
69 task automatic drive_bit;
70     input b;
71     begin
72         line = b;
73         hold_ticks(0S);
74     end
75 endtask
76
77 task automatic clear_flags;
78     begin
79         seen_e = 1'b0; seen_o = 1'b0;
80         cpe_e = 1'b0; cfe_e = 1'b0;
81         cpe_o = 1'b0; cfe_o = 1'b0;
82     end
83 endtask
84
85 task automatic drive_frame;
86     input [`UART_DATA_BITS-1:0] d;
87     input par;
88     input stop;
89     integer k;
90     begin
91         drive_bit(1'b0);
92         drive_bit(par);
93         for (k = `UART_DATA_BITS-1; k >= 0; k = k - 1) drive_bit(d[k]);
94         drive_bit(stop);
95     end
96 endtask
97
98 initial begin
99     clear_flags;
100     @(negedge Clk); RstN = 1'b0;
101     repeat (4) @(negedge Clk); RstN = 1'b1;
102     hold_ticks(2);
103
104     chk("idle after reset", bsy_e === 1'b0 && bsy_o === 1'b0);
105
106     for (c = 0; c < 128; c = c + 1) begin
107         clear_flags;
108         drive_frame(c[6:0], ^c[6:0], 1'b1);
109         hold_ticks(2);
110         chk("even: frame accepted", seen_e);
111         chk("even: register value", cap_e === {^c[6:0], c[6:0]});
112         chk("even: no parity error", cpe_e === 1'b0);
113         chk("even: no frame error", cfe_e === 1'b0);
114         chk("even: data output", dat_e === c[6:0]);
115         chk("odd DUT flags parity", seen_o && cpe_o === 1'b1);
116
117         clear_flags;
118         drive_frame(c[6:0], ~(^c[6:0]), 1'b1);
119         hold_ticks(2);
120         chk("odd: frame accepted", seen_o);
121         chk("odd: register value", cap_o === {~(^c[6:0]), c[6:0]});

```

```

122     chk("odd: no parity error", cpe_o == 1'b0);
123     chk("even DUT flags parity", seen_e && cpe_e == 1'b1);
124 end
125
126 clear_flags;
127 drive_frame(7'h41, ~(^7'h41), 1'b1);
128 hold_ticks(2);
129 chk("bad parity reported", seen_e && cpe_e == 1'b1);
130 chk("data still captured", cap_e[UART_DATA_BITS-1:0] == 7'h41);
131
132 clear_flags;
133 drive_frame(7'h55, ^7'h55, 1'b0);
134 hold_ticks(2);
135 chk("framing error reported", seen_e && cfe_e == 1'b1);
136 line = 1'b1; hold_ticks(4);
137 clear_flags;
138 drive_frame(7'h2A, ^7'h2A, 1'b1);
139 hold_ticks(2);
140 chk("recovers after frame error", seen_e && cap_e == {^7'h2A, 7'h2A});
141 chk("flags cleared on good frame", cfe_e == 1'b0 && cpe_e == 1'b0);
142
143 clear_flags;
144 line = 1'b0; hold_ticks(OS/4);
145 line = 1'b1; hold_ticks(2*OS);
146 chk("glitch rejected", !seen_e && !seen_o && bsy_e == 1'b0);
147
148 clear_flags;
149 drive_frame(7'h7F, ^7'h7F, 1'b1);
150 chk("first of a burst", seen_e && cap_e == {^7'h7F, 7'h7F});
151 clear_flags;
152 drive_frame(7'h00, ^7'h00, 1'b1);
153 chk("second of a burst", seen_e && cap_e == {^7'h00, 7'h00});
154 clear_flags;
155 drive_frame(7'h3C, ^7'h3C, 1'b1);
156 hold_ticks(2);
157 chk("third of a burst", seen_e && cap_e == {^7'h3C, 7'h3C});
158
159 $display("tb_uart_rx: Passed: %0d, Failed: %0d", pass, fail);
160 if (fail != 0) $fatal(1);
161 $finish;
162 end
163 endmodule

```

تست لینک دو سناریو دارد. در اولی دو واحد کاملاً هم‌نرخ‌اند و هر ۱۲۸ کاراکتر هم‌زمان در هر دو جهت رد و بدل می‌شود (واحد A کاراکتر و واحد B مکمل بیتی همان کاراکتر را می‌فرستد تا دو جهت هرگز الگوی یکسان حمل نکنند).

در دومی نسبت تقسیم دو واحد ۲۰ و ۲۱ گرفته شده است، یعنی اختلاف نرخ بیت ۵٪. این عدد اتفاقی نیست؛ درست زیر حد نظری قرار دارد. آخرین بیتی که گیرنده نمونه می‌گیرد بیت خاتمه است که ۹/۵ بیت پس از نقطه‌ی هم‌گامی قرار دارد و نمونه تا وقتی درست است که رانش انباشته از نصف بیت بیشتر نشود:

$$\text{حد اختلاف نرخ} = \frac{0.5}{9.5} \approx 5.3\%$$

این حد به‌صورت تجربی هم اندازه‌گیری شد: با جاروب چند لینک با اختلاف‌های مختلف، تا ۵٪ همه‌ی کاراکترها سالم می‌رسند و از ۶٪ به بعد هیچ‌کدام نمی‌رسند. یعنی مدار واقعا به همان حدی می‌رسد که تئوری می‌گوید، نه کمتر.

```

1 `timescale 1ns/1ps
2 `include "uart_pkg.vh"
3
4 module tb_uart_link;
5     localparam integer OS = `UART_OVERSAMPLE;
6
7     reg Clk = 1'b0, RstN = 1'b1;
8     always #5 Clk = ~Clk;
9
10    reg a_start = 1'b0, b_start = 1'b0;
11    reg [UART_DATA_BITS-1:0] a_data = 7'd0, b_data = 7'd0;
12    reg s_astart = 1'b0, s_bstart = 1'b0;
13    reg [UART_DATA_BITS-1:0] s_adata = 7'd0, s_bdata = 7'd0;
14
15    wire [UART_RX_REG_BITS-1:0] ok_arx, ok_brx;
16    wire ok_avld, ok_bvld, ok_ape, ok_afe, ok_bpe, ok_bfe;
17    wire ok_abusy, ok_bbussy, ok_adone, ok_bdone, ok_ab, ok_ba;
18
19    uart_link #(A_OS_DIVISOR(3), B_OS_DIVISOR(3)) link_ok (

```

```

20 .Clk(Clk), .RstN(RstN),
21 .a_tx_start(a_start), .a_tx_data(a_data),
22 .a_tx_busy(ok_abusy), .a_tx_done(ok_adone),
23 .a_rx_reg(ok_arx), .a_rx_data(), .a_rx_valid(ok_avld),
24 .a_parity_err(ok_ape), .a_frame_err(ok_afe),
25 .b_tx_start(b_start), .b_tx_data(b_data),
26 .b_tx_busy(ok_bbusy), .b_tx_done(ok_bdone),
27 .b_rx_reg(ok_brx), .b_rx_data(), .b_rx_valid(ok_bvld),
28 .b_parity_err(ok_bpe), .b_frame_err(ok_bfe),
29 .a_to_b(ok_ab), .b_to_a(ok_ba)
30 );
31
32 wire [`UART_RX_REG_BITS-1:0] sk_arx, sk_brx;
33 wire sk_avld, sk_bvld, sk_ape, sk_afe, sk_bpe, sk_bfe, sk_abusy, sk_bbusy;
34
35 uart_link #(.A_OS_DIVISOR(20), .B_OS_DIVISOR(21)) link_skew (
36 .Clk(Clk), .RstN(RstN),
37 .a_tx_start(s_astart), .a_tx_data(s_adata),
38 .a_tx_busy(sk_abusy), .a_tx_done(),
39 .a_rx_reg(sk_arx), .a_rx_data(), .a_rx_valid(sk_avld),
40 .a_parity_err(sk_ape), .a_frame_err(sk_afe),
41 .b_tx_start(s_bstart), .b_tx_data(s_bdata),
42 .b_tx_busy(sk_bbusy), .b_tx_done(),
43 .b_rx_reg(sk_brx), .b_rx_data(), .b_rx_valid(sk_bvld),
44 .b_parity_err(sk_bpe), .b_frame_err(sk_bfe),
45 .a_to_b(), .b_to_a()
46 );
47
48 integer pass = 0, fail = 0;
49 integer c;
50
51 reg okA_seen, okB_seen, skA_seen, skB_seen;
52 reg [`UART_RX_REG_BITS-1:0] okA_reg, okB_reg, skA_reg, skB_reg;
53 reg okA_err, okB_err, skA_err, skB_err;
54
55 always @(posedge Clk) begin
56     if (ok_avld) begin okA_seen <= 1'b1; okA_reg <= ok_arx; okA_err <= ok_ape | ok_afe; end
57     if (ok_bvld) begin okB_seen <= 1'b1; okB_reg <= ok_brx; okB_err <= ok_bpe | ok_bfe; end
58     if (sk_avld) begin skA_seen <= 1'b1; skA_reg <= sk_arx; skA_err <= sk_ape | sk_afe; end
59     if (sk_bvld) begin skB_seen <= 1'b1; skB_reg <= sk_brx; skB_err <= sk_bpe | sk_bfe; end
60 end
61
62 task automatic chk;
63     input [255:0] name;
64     input ok;
65     begin
66         if (ok) pass = pass + 1;
67         else begin
68             fail = fail + 1;
69             $display(" FAIL @%0t: %0s", $time, name);
70         end
71     end
72 endtask
73
74 task automatic exchange;
75     input [`UART_DATA_BITS-1:0] da;
76     input [`UART_DATA_BITS-1:0] db;
77     begin
78         wait (!ok_abusy && !ok_bbusy);
79         okA_seen = 1'b0; okB_seen = 1'b0; okA_err = 1'b0; okB_err = 1'b0;
80         @(negedge Clk); a_data = da; b_data = db; a_start = 1'b1; b_start = 1'b1;
81         @(negedge Clk); a_start = 1'b0; b_start = 1'b0;
82         wait (okA_seen && okB_seen);
83         @(negedge Clk);
84     end
85 endtask
86
87 task automatic exchange_skew;
88     input [`UART_DATA_BITS-1:0] da;
89     input [`UART_DATA_BITS-1:0] db;
90     begin
91         wait (!sk_abusy && !sk_bbusy);
92         skA_seen = 1'b0; skB_seen = 1'b0; skA_err = 1'b0; skB_err = 1'b0;
93         @(negedge Clk); s_adata = da; s_bdata = db; s_astart = 1'b1; s_bstart = 1'b1;
94         @(negedge Clk); s_astart = 1'b0; s_bstart = 1'b0;
95         wait (skA_seen && skB_seen);
96         @(negedge Clk);
97     end
98 endtask
99
100 initial begin
101     okA_seen = 1'b0; okB_seen = 1'b0; skA_seen = 1'b0; skB_seen = 1'b0;
102     okA_err = 1'b0; okB_err = 1'b0; skA_err = 1'b0; skB_err = 1'b0;
103
104     @(negedge Clk); RstN = 1'b0;
105     repeat (4) @(negedge Clk); RstN = 1'b1;
106     repeat (4) @(negedge Clk);

```

```

107     chk("both lines idle high", ok_ab === 1'b1 && ok_ba === 1'b1);
108
109     for (c = 0; c < 128; c = c + 1) begin
110         exchange(c[6:0], ~c[6:0]);
111         chk("B received A's character", okB_reg === {~c[6:0], c[6:0]});
112         chk("A received B's character", okA_reg === {~c[6:0], ~c[6:0]});
113         chk("no errors on either end", !okA_err && !okB_err);
114     end
115
116     wait (!ok_abusy);
117     okB_seen = 1'b0;
118     @(negedge Clk); a_data = 7'h48; a_start = 1'b1;
119     @(negedge Clk); a_start = 1'b0;
120     @(posedge ok_adone);
121     @(negedge Clk); a_data = 7'h69; a_start = 1'b1;
122     @(negedge Clk); a_start = 1'b0;
123     wait (okB_seen); chk("burst 'H'", okB_reg[UART_DATA_BITS-1:0] === 7'h48);
124     okB_seen = 1'b0;
125     wait (okB_seen); chk("burst 'i'", okB_reg[UART_DATA_BITS-1:0] === 7'h69);
126     wait (!ok_abusy && !ok_bbusy);
127
128     chk("both units idle at the end", !ok_abusy && !ok_bbusy);
129
130     for (c = 0; c < 16; c = c + 1) begin
131         exchange_skew({c[3:0], 3'b101}, {3'b010, c[3:0]});
132         chk("skewed link: B received A", skB_reg === {~{c[3:0], 3'b101}, c[3:0], 3'b101});
133         chk("skewed link: A received B", skA_reg === {~{3'b010, c[3:0]}, 3'b010, c[3:0]});
134         chk("skewed link: no errors", !skA_err && !skB_err);
135     end
136
137     $display("tb_uart_link: Passed: %0d, Failed: %0d", pass, fail);
138     if (fail != 0) $fatal(1);
139     $finish;
140 end
141 endmodule
142

```

اندازه‌گیری نقطه‌ی نمونه‌برداری

ادعای «نمونه‌برداری از وسط بیت» چیزی است که به راحتی می‌شود نوشت و اشتباه پیاده‌سازی کرد، چون یک نمونه‌ی کمی دور از وسط هم در لینک هم‌نرخ کاملاً درست کار می‌کند و هیچ تستی را رد نمی‌کند. تنها اثرش این است که حاشیه‌ی تحمل اختلاف نرخ بی‌سروصدا کم می‌شود.

بنابراین نقطه‌ی نمونه‌برداری مستقیماً اندازه‌گیری شد: شمارش پالس‌های `os_tick` از لحظه‌ی پایین آمدن خط تا لحظه‌ای که هر بیت در ثبات می‌نشیند. نقاط ایده‌آل ۸، ۲۴، ۴۰، ... هستند.

اولین اندازه‌گیری اعداد ۲۶، ۴۲، ۵۸، ... را داد؛ یعنی هر نمونه دو تیک دیرتر از وسط بیت گرفته می‌شد. علتش یک خطای واحد در شمارنده‌ی نیم‌بیت بود:

```

if (os == OS/2) // wrong: os starts at 0 and is tested BEFORE
                // incrementing, so this waits OS/2 + 1 ticks
if (os == OS/2 - 1) // right: counting 0..OS/2-1 is exactly OS/2 ticks

```

پس از اصلاح، اعداد به ۲۵، ۴۱، ۵۷، ... رسیدند. یک تیک باقی‌مانده اجتناب‌ناپذیر است: شمارنده‌ی داخل بیت فقط روی مرز پالس‌های نمونه‌برداری می‌تواند شروع شود، پس تا $\frac{1}{2}$ بیت خطای اولیه همیشه وجود دارد — همان چیزی که در تحلیل حاشیه‌ی خطا هم لحاظ شده است.

اثر این یک تیک کم نیست: پیش از اصلاح، لینک با اختلاف نرخ ۵٪ کاراکتر گم می‌کرد و اکنون نمی‌کند.

نتایج تست

```

iverilog: Icarus Verilog version 12.0 (stable) ()

=== tb_uart_tx ===
tb_uart_tx: Passed: 4108, Failed: 0
PASS: tb_uart_tx

=== tb_uart_rx ===
tb_uart_rx: Passed: 1290, Failed: 0

```

```
PASS: tb_uart_rx
=== tb_uart_link ===
tb_uart_link: Passed: 436, Failed: 0
PASS: tb_uart_link
summary: 3/3 passed, 0 failed
```

خروجی سنتز (Quartus Prime)

طرح روی Intel Quartus Prime Lite Edition 21.1.0 با موجودیت سطح بالای uart_link (دو نمونه‌ی uart در uart_link) سنتز و Route & Place شد. دستگاه هدف: Cyclone V (5CGXFC7C7F23C8).

مورد	نتیجه
وضعیت کامپایل	موفق (۰ خطا، ۱۴ هشدار)
Logic utilization (ALMs)	۹۶/۵۶۴۸۰ (< ۱٪)
Total registers	۱۵۰
Total pins	۶۰/۲۶۸ (۲۲٪)
• Block memory bits	
• DSP Blocks	
Worst-case setup slack	-۰/۶۷۴
Worst-case hold slack	+۰/۱۶۶
Min pulse width slack	-۰/۰۹۱

